Assignment 4: Heap Data Structures: Implementation, Analysis, and Applications

Stephanie Peguero

1. Heapsort Implementation and Analysis

Theoretical Analysis

This assignment implements Heapsort, a comparison-based sorting algorithm that uses a max-heap data structure to sort an array in place. Based on Chapter 6 of the textbook, the time complexity of this algorithm is determined by its two primary procedures: BUILD-MAX-HEAP and MAX-HEAPIFY.

- **Worst-Case Time Complexity ($O(n \log n)$):** The BUILD-MAX-HEAP procedure organizes the initial array into a valid max-heap in $O(n)$ time. Next, the HEAPSORT procedure loops $n - 1$ times. In each iteration, it swaps the largest element (the root) to the end of the array and calls MAX-HEAPIFY to push the new root down to its proper place.
    - **Recurrence Relation:** The $O(\log n)$ complexity of the MAX-HEAPIFY procedure can be formally proven using a recurrence relation. The worst-case running time $T(n)$ occurs when the tree is unbalanced, making the recursive call on a child's subtree of size at most $2n/3$. The time to fix the relationship between the node and its children is $\Theta(1)$. This gives the recurrence relation: $T(n) \leq T(2n/3) + \Theta(1)$. According to Case 2 of the Master Theorem (from Chapter 4), the solution to this recurrence is $T(n) = O(\log n)$.
    - Therefore, n calls to MAX-HEAPIFY take $O(n \log n)$ time, giving a total worst-case complexity of $O(n) + O(n \log n) = O(n \log n)$.
- **Average and Best-Case Time Complexity ($O(n \log n)$):** Unlike some algorithms that run faster on sorted data, Heapsort is structurally rigid. Even if the array is already sorted, the algorithm must tear down that order to build the max-heap, and it must still extract elements one by one, shifting the remaining elements down the tree. Thus, the best, average, and worst-case time complexities are all mathematically guaranteed to be $O(n \log n)$.

- **Space Complexity:** A major benefit of Heapsort is that it sorts the data *in-place*. The algorithm manages the tree structure entirely through index math rather than creating new objects, meaning it requires only $O(1)$ auxiliary space.

Empirical Comparison

To test the theoretical analysis, the Heapsort implementation was benchmarked against Merge Sort and Randomized Quicksort across four different array distributions of size $n = 1000$.

Distribution	Heapsort	Merge Sort	Quicksort
Random Array	0.00525s	0.00276s	0.00174s
Sorted Array	0.00338s	0.00116s	0.00157s
Reverse Sorted Array	0.00230s	0.00395s	0.00331s
Repeated Array	0.00246s	0.00144s	0.00018s

Discussion of Results: The empirical data demonstrates that Heapsort exhibits a highly reliable, consistent time complexity. While Randomized Quicksort's execution steps depend heavily on the initial order of the elements (which can degrade to $O(n^2)$ if a poor pivot is repeatedly chosen), Heapsort's performance is strictly governed by its tree structure. This structural guarantee explains why Heapsort's execution times remained remarkably stable across all four dataset distributions.

However, despite all three algorithms sharing an $O(n \log n)$ expected time complexity, there are notable differences in their real-world speeds due to the constant factors hidden by Big-O notation. Quicksort generally outperformed Heapsort in practice because, as the textbook notes, it has tight inner loops, which results in a much smaller hidden constant factor in its running time (Cormen et al., 2022, p. 159). Furthermore, Quicksort was blisteringly fast on the "Repeated Array" (0.00018s) because the implementation carried over from previous analyses uses a three-way partition that isolates duplicates, allowing it to skip unnecessary work. Ultimately, while Quicksort and Merge Sort are faster in absolute terms, Heapsort provides an incredibly valuable combination of guaranteed $O(n \log n)$ speed and $O(1)$ memory efficiency.

2. Priority Queue Implementation and Applications

Data Structure Justification

For the second part of the assignment, I designed a task scheduler simulation using a Binary Max-Heap. This heap is backed by a standard Python list, which is highly efficient because it

avoids the need to build complex tree node objects linked by pointers. Instead, the relatives of any task can be found using simple $O(1)$ arithmetic on its index i: the parent is at $(i - 1)//2$, the left child is at $2i + 1$, and the right child is at $2i + 2$.

To manage the tasks, I created a Task class containing a task_id, priority, arrival_time, and deadline. A major challenge in implementing a priority queue is updating a task's priority quickly. If the program had to search the whole array to find a specific task, the operation would take $O(n)$ time. To solve this, the MaxPriorityQueue class includes a Python dictionary (a hash map) that tracks the exact array index of every task. Whenever a task is swapped in the heap, its index is updated in the dictionary.

Impact of Load Factor: While the hash map provides expected $O(1)$ time lookups, this performance relies on maintaining a low load factor ($\alpha = n/m$, the ratio of elements to available buckets). If the load factor becomes too high, collisions increase, and the lookup time degrades toward $O(n)$. If the dictionary lookup degraded to $O(n)$, the overall time complexity of the increase_key operation would also degrade to $O(n)$, entirely defeating the purpose of the heap. To prevent this degradation, dynamic resizing is utilized. As established in previous hash table analyses, whenever the load factor exceeds a certain threshold, the underlying array is reallocated with double the capacity to maintain a low load factor, ensuring operations remain $O(1)$ in amortized time (Miller & Ranum, 2011). This guarantees that task indices can always be located rapidly, preserving the strict logarithmic bounds of the heap operations.

Core Operations Analysis

Because the maximum height of a heap is log n, the core operations only ever have to traverse a single path from the root to a leaf (or vice versa).

- **insert(task):** To insert a new task, the method places the element at the very end of the array, assigns it a priority of negative infinity, and then immediately calls increase_key to float it up to its correct position. The time complexity is $O(\log n)$.
- **extract_max():** This method retrieves the highest priority task at the root (index 0). To fill the empty root, it moves the very last leaf to the top of the tree, shrinks the active heap size by one, and calls MAX-HEAPIFY to float the new root down to its proper place. The time complexity is $O(\log n)$.

- **increase_key(task, new_priority):** Thanks to the index-tracking dictionary (with its low load factor ensuring $O(1)$ amortized lookups), finding the task takes $O(1)$ time. Once the priority is updated, the task is compared to its parent and swapped upwards until the max-heap property is restored. Since the path to the root is at most $\log n$ nodes long, the time complexity is $O(\log n)$.

- **is_empty():** Checking if the queue is empty simply involves checking the length of the underlying list, which takes $O(1)$ time.

References

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms* (4th ed.). MIT Press.

Miller, B. N., & Ranum, D. L. (2011). *Problem solving with algorithms and data structures using Python* (2nd ed.). Franklin, Beedle & Associates.